CS471 Project Proposal: PolyRack-Beyond Hotness

Tiered Memory Management Beyond Hotness

Mayul Besnard
EPFL
Lausanne, Switzerland
mayeul.besnard@epfl.ch

Sergio Boffi
EPFL
Lausanne, Switzerland
sergio.boffi@epfl.ch

***Abstract*—Tiered memory systems combine a fast tier (local DRAM) with a larger but slower tier (e.g., CXL-attached memory) and typically decide data placement based on access frequency ("hotness"). Liu et al. [?] argue that hot data is not necessarily performance-critical, because memory-level parallelism (MLP) can hide most of the slow-tier latency of some accesses but not of others. They introduce the Amortized Offcore Latency (AOL) metric and build two mechanisms on top of it: SOAR, an AOL-guided first-placement policy, and ALTO, an AOL-based filter for page migrations. In this project we will reproduce the main claims of the paper (hotness is a poor proxy of performance, AOL is a better predictor of slowdown, SOAR and ALTO outperform state-of-the-art tiering systems) on a traditional server and, if time permits, evaluate whether these techniques also improve memory management on the PolyRack disaggregated rack architecture [?].**

I. Introduction and Problem Statement

A. Why is the problem important?

Server performance is directly tied to the number of instructions executed per cycle: Perf $\propto$ IPC $= 1/$CPI. Every cycle the core spends stalled waiting for memory increases the CPI and therefore degrades performance. Memory-intensive datacenter workloads (graph analytics, machine learning, caching, HPC) keep growing in footprint, and provisioning all of their memory in DRAM is expensive. Tiered memory architectures therefore add a cheaper, higher-capacity slow tier (CXL memory, remote NUMA node) next to the fast DRAM tier. The slow tier, however, is 2–3$\times$ slower than DRAM [**?**], so the way data is distributed across tiers determines how many memory stalls the application suffers: placing too much performance-critical data in the slow tier, or migrating pages too aggressively, both translate into lost performance.

B. What is involved in the problem?

The problem involves (i) the two memory tiers and their latencies T_{DRAM} and T_{CXL}, (ii) a notion of data *hotness*, i.e., how frequently a page or object is accessed, (iii) the *initial placement* policy that decides in which tier data is allocated, and (iv) the *migration* policy that promotes pages to the fast tier or demotes them to the slow tier (and possibly re-migrates them) at run time.

A simple first-order model makes the dependencies explicit. Let α be the fraction of data placed in DRAM. The average memory latency is

$$T_{\text{total}} = \alpha\, T_{\text{DRAM}} + (1-\alpha)\, T_{\text{CXL}}. \tag{1}$$

Let $\beta_{\text{DRAM,hot}}$ ($\beta_{\text{CXL,hot}}$) be the fraction of hot data in DRAM (CXL). Splitting each tier into hot and cold data gives

$$\begin{aligned} T_{\text{total}} = {} & \underbrace{\alpha\,\beta_{\text{DRAM,hot}} T_{\text{DRAM}}}_{\text{hot in DRAM}} + \underbrace{\alpha\,(1-\beta_{\text{DRAM,hot}}) T_{\text{DRAM}}}_{\text{cold in DRAM}} \\ & + \underbrace{(1-\alpha)\,\beta_{\text{CXL,hot}} T_{\text{CXL}}}_{\text{hot in CXL}} + \underbrace{(1-\alpha)(1-\beta_{\text{CXL,hot}}) T_{\text{CXL}}}_{\text{cold in CXL}}. \end{aligned} \tag{2}$$

State-of-the-art (SOTA) tiering systems essentially try to move data from the "hot in CXL" term to the "hot in DRAM" term, i.e., to decrease $\beta_{\text{CXL,hot}}$ and increase $\beta_{\text{DRAM,hot}}$.

C. What causes the problem?

Two factors cause poor tiering decisions. First, both slow-tier accesses and page migrations are expensive (up to μs-scale for migrations), so wrong decisions are costly. Second, and more fundamentally, the decisions are based on hotness, which ignores *memory-level parallelism*. Out-of-order cores overlap independent memory requests, so the effective cost of an access is roughly its latency divided by the degree of MLP. Let $\gamma_{\text{MLP,fast}}$ be the fraction of data accessed with high MLP (e.g., array traversals); the remaining $1-\gamma_{\text{MLP,fast}}$ is accessed with low MLP (e.g., dependent pointer chasing). Writing $\beta = \beta_{\text{DRAM,hot}}$ and $\gamma = \gamma_{\text{MLP,fast}}$, the DRAM part of the model becomes

$$\begin{aligned} T_{\text{total}} = {} & \alpha\,\beta\,\gamma\,\frac{T_{\text{DRAM}}}{\text{MLP}_{\text{fast}}} + \alpha\,\beta\,(1-\gamma)\frac{T_{\text{DRAM}}}{\text{MLP}_{\text{slow}}} \\ & + \alpha(1-\beta)\,\gamma\,\frac{T_{\text{DRAM}}}{\text{MLP}_{\text{fast}}} + \alpha(1-\beta)(1-\gamma)\frac{T_{\text{DRAM}}}{\text{MLP}_{\text{slow}}} \\ & + \dots \end{aligned} \tag{3}$$

and analogously for the CXL terms. Hot data is often accessed with high MLP (sequential scans), so its latency is already largely hidden even in the slow tier, while "colder" but low-MLP data pays the full slow-tier latency. When a hotness-based policy reduces $\beta_{\text{CXL,hot}}$, it fills DRAM with high-MLP data and pushes low-MLP data to CXL: the low-MLP terms grow, CPI increases, and performance drops. Hotness alone is therefore an unfaithful proxy of performance.

D. Does prior work solve the problem?

No. SOTA tiering systems such as TPP, Nomad, Linux NUMA Balancing Tiering (NBT) and Colloid all rely on

Algorithm 1 ALTO: AOL-regulated page migrations (e.g., every 1 s) [**?**]

Require: $\text{AOL}_{\text{low}} \leftarrow 40$, $\text{AOL}_{\text{high}} \leftarrow 100$ ▷ profiled offline
Ensure: Adjusted page migration ratio *Scale*

```
Profile the current AOL (l)
if l ≤ AOL_low then
    Scale ← 0                          ▷ no promotions
else if l ≥ AOL_high then
    Scale ← 1                          ▷ all promotions
else
    Scale ← F(l)                       ▷ partial promotions
end if
```

hotness. On the paper's microbenchmark (one sequential, high-MLP "hot" thread and one pointer-chasing, low-MLP "cold" thread), they underperform even a naive *Cold-on-DRAM* placement, and fall behind first-touch allocation without migrations (NoTier) by 12–14% [**?**, Fig. 1]. On real workloads such as `bc-urand` (GAPBS betweenness centrality on a uniform random graph), they can be up to 217% slower than NoTier [**?**, Fig. 4].

*E. Overview of the proposed solution [**?**]*

AOL. The paper first shows (via an offline study of 56 workloads) that the ratio of LLC stall cycles to total cycles, $P = s_{\text{LLC}}/c$, is correlated with the slowdown on the slow tier, but overestimates it for high-MLP workloads. It then defines $\text{AOL} = \text{Latency}/\text{MLP}$, computed from offcore performance counters, and models the slowdown as $S = P \times K$ with a correction factor $K = f(\text{AOL}) = 1/(a + b/\text{AOL})$. The constants a, b depend only on the hardware and can be calibrated with microbenchmarks; the Pearson correlation between predicted and measured slowdown improves from 0.869 to 0.951.

SOAR (Static Object Allocation based on Ranking) profiles an application, ranks objects by their AOL-weighted contribution to performance, and places the highest-ranked objects in DRAM at allocation time. This yields near-optimal first placement and removes the need for most later migrations.

ALTO observes that automatic hot-page migration often does not improve performance and regulates migrations with the current AOL l (Alg. **??**): below AOL_{low} promotions are disabled, above AOL_{high} they are all enabled, and in between only a fraction $F(l)$ is allowed. ALTO can be plugged into existing systems (TPP, NBT, Nomad, Colloid).

F. Project goals

With this project we aim to:

- reproduce the main results of "Tiered Memory Management Beyond Hotness" [**?**]: the failure of hotness-based metrics, the AOL-based slowdown predictor, and the gains of SOAR and ALTO over SOTA tiering systems;
- if time permits, test AOL, SOAR and ALTO on the PolyRack architecture [**?**] and quantify their benefit there.

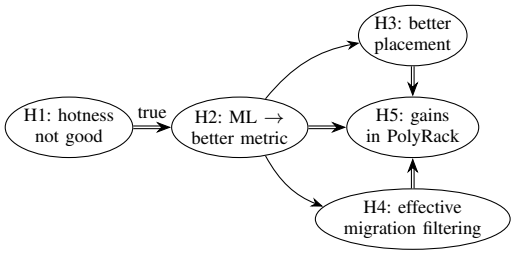


Fig. 1. Hypothesis tree. H2–H5 are only meaningful if H1 holds.

II. Questions, Hypotheses and Tests

The overarching question is: *how can we improve server performance by reducing effective memory latency, using metrics better than hotness and building better data-placement procedures on top of them?* We break it down into the following prioritized questions; their dependencies are summarized in the hypothesis tree of Fig. **??**.

A. Q1: Are hotness metrics really informative?

- **Insight (I1):** MLP hides part of the memory latency, so simple hotness-based decisions lead to suboptimal system performance.
- **Hypothesis (H1):** hotness $\neq$ performance criticality.
- **Test (E1):** reproduce the hot/cold microbenchmark (sequential vs. pointer-chasing threads) and evaluate the performance of different placement strategies (All-on-DRAM, Hot-on-DRAM, Cold-on-DRAM, NoTier, SOTA systems), cf. Fig. 1 of [**?**].

B. Q2: If not, how can a better metric be defined?

- **Insight (I2):** ML/data-driven exploration of hardware counters across many workloads helps find which features actually explain slowdown.
- **Hypothesis (H2):** an ML-derived metric (AOL with correction factor K) is more representative of performance than hotness or raw LLC stalls.
- **Test (E2):** measure slowdown and counter features (LLC stalls/cycle, AOL) on a set of workloads, calibrate $K = f(\text{AOL})$, and compare the Pearson coefficient between measured slowdown and the predictors P and $P \times K$, cf. Fig. 2 of [**?**].

C. Q3: With this metric, can we improve first data placement (SOAR)?

- **Insight (I3):** AOL improves first placement by correctly identifying performance-critical data.
- **Hypothesis (H3):** an AOL-based placement policy performs better than SOTA.
- **Test (E3):** compare the slowdown (w.r.t. All-on-DRAM) of SOAR vs. NoTier, Colloid, Nomad, NBT (and TPP) on different workloads and fast/slow-tier ratios, cf. Fig. 4 and Table 4 of [**?**].

D. Q4: With this metric, can we optimize page migration policies (ALTO)?

- **Insight (I4):** AOL allows filtering page migrations depending on their true performance criticality.
- **Hypothesis (H4):** SOTA + ALTO page filtering performs better than SOTA alone.
- **Test (E4):** compare slowdown, effective memory latency and number of page promotions of SOTA+ALTO vs. SOTA alone, cf. Fig. 7 and Table 5 of [**?**].

E. Q5: Can these elements improve PolyRack?

- **Insight (I5):** AOL, SOAR and ALTO increase performance on tiered-memory servers; PolyRack disaggregates memory into dedicated memory servers connected through a CXL-class fabric, which creates a similar fast/slow memory hierarchy.
- **Hypothesis (H5):** AOL/SOAR/ALTO also lead to better performance on PolyRack.
- **Test (E5):** compare slowdown and memory latency on PolyRack with vs. without AOL/SOAR/ALTO on different workloads, and against a traditional tiered server.

III. Evaluation Methodology

A. Experimental setup

We will first reproduce the results on a traditional server, and only then move to PolyRack. Following [**?**], if no real CXL device is available the slow tier will be emulated with a remote NUMA node (lowering its uncore frequency and disabling its cores, which in the paper yields 90/190 ns fast/slow latencies). We will use the open-source implementation of SOAR and ALTO (https://github.com/MoatLab/SoarAlto), Linux `perf` and the offcore/LLC performance counters needed to compute P and AOL, and `numactl`/cgroups to control the fast/slow-tier ratio. [TODO: Confirm server access, CPU/kernel version, CXL vs. NUMA emulation, and whether experiments run on bare metal or in a VM.]

B. Workloads

We will start from `bc-urand` and other graph workloads from the GAP Benchmark Suite (GAPBS), which stress tiered memory with irregular accesses, plus the hot/cold microbenchmark of Q1. Time permitting, we will add workloads from the other categories used in the paper (machine learning, caching, HPC). [TODO: Agree on the final workload list with the TA (and on PolyRack's home workloads).]

C. Baselines and metrics

Baselines: All-on-DRAM (upper bound), NoTier (first-touch, no migrations), and the hotness-based SOTA systems TPP, NBT, Nomad and Colloid. Metrics: slowdown relative to All-on-DRAM (lower is better), Pearson correlation between predicted and measured slowdown, AOL and average memory latency, and migration traffic (number of page promotions/demotions), all measured across several fast/slow-tier ratios (e.g., 10–90% of the workload's RSS in the slow tier).

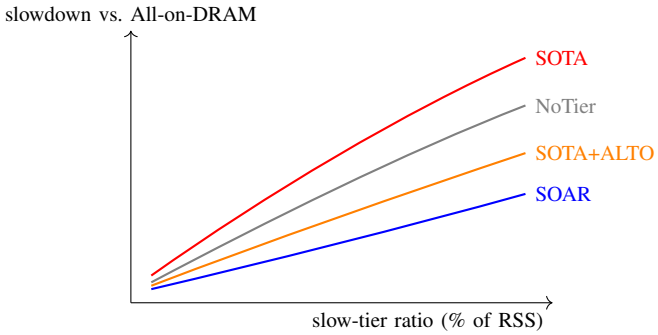


Fig. 2. Sketch of the expected main result: AOL-based policies keep the slowdown lower than hotness-based SOTA and NoTier across slow-tier ratios.

IV. Final Deliverable

A. Concrete goals

By the end of the semester we want to (1) have a working tiered-memory setup with the baselines and SOAR/ALTO running, (2) confirm or refute H1–H4 on our hardware, and (3) if time permits, provide a first evaluation of H5 on PolyRack.

B. Expected results

Our key figure (Fig. **??**) will plot the slowdown w.r.t. All-on-DRAM (Y-axis) as a function of the fraction of the working set placed in the slow tier (X-axis), for NoTier, a hotness-based SOTA system, SOTA+ALTO and SOAR. We expect slowdown to grow with the slow-tier ratio for all policies, with SOTA sometimes worse than NoTier (due to useless migrations) and SOAR and SOTA+ALTO consistently below both. We will also show the predicted-vs.-measured slowdown scatter plot for P and $P \times K$ (Q2).

V. Milestones

1) **Oct. 13:** proposal and presentation; study [**?**] and get access to the servers.
2) **Late Oct.:** set up the environment (tiered/NUMA-emulated memory, SOAR/ALTO code, baselines) and reproduce the microbenchmark (Q1).
3) **Nov. 12 (status report):** AOL metric calibration and slowdown prediction (Q2); first SOAR results on `bc-urand` (Q3).
4) **End of Nov.:** full SOAR and ALTO evaluation vs. SOTA on the selected workloads (Q3, Q4).
5) **Early Dec.:** (if time permits) PolyRack extension (Q5).
6) **Dec. 10 / Dec. 17 / Dec. 20:** poster, poster session and final report.

VI. Open Questions

1) Will we test AOL/SOAR/ALTO directly on PolyRack, or on a traditional server beforehand? (Our plan assumes the latter.)
2) Which home workloads will we work with?
3) How does access to the servers work?
4) How will we set up our experiments (programming language, bare metal vs. VM, etc.)?

REFERENCES

[1] J. Liu, H. Hadian, H. Xu, and H. Li, "Tiered Memory Management Beyond Hotness," in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Boston, MA, USA, Jul. 2025.

[2] P. Poorsarvi Tehrani, S. Lin, A. Ning, B. Falsafi, D. Atienza, T. Bourgeat, and S. Kashyap, "PolyRack: Servers for the Post-Moore Era," EPFL, 2026.